

ASSIGNMENT 8

Instructions for Students

- Create a separate HTML and CSS file for each question.
- Do not use JavaScript or any external CSS frameworks.
- Use semantic HTML5 tags (header, main, section, footer, etc.).
- Keep HTML and CSS neat, well-indented, and properly commented.
- Use only Flexbox or Grid for layout alignment where required.
- Maintain proper contrast, spacing, and responsive behavior.

1. Create a Webpage Demonstrating Pseudo-Classes

Requirements:

- Show examples of pseudo-classes such as :hover, :focus, :active, and :nth-child().
- Apply these selectors on links, buttons, and list items.
- Use color or background changes to show state differences.
- Include short text explaining each pseudo-class beside its example.
- Use Flexbox to align all examples neatly in one row.

2. Design a Tooltip Demonstration Page

Requirements:

- Create multiple elements (like icons or words) that display tooltips on hover.
- Build tooltips using only CSS (position: absolute; and opacity transitions).
- Each tooltip should fade in smoothly using transition: 0.3s ease-in-out.
- Ensure tooltips remain readable and positioned above the hovered element.
- Center all examples on the page using Flexbox alignment.

3. Create an Image Showcase Layout

Requirements:

- Display six images arranged using Grid or Flexbox.
- On hover, each image should slightly scale up (transform: scale(1.1)) and darken using a semi-transparent overlay.
- Add rounded corners and box-shadow to make images modern.
- Include short captions below each image using <figcaption>.
- Make the layout responsive and stack images in one column below 700px.

4. **Build a CSS Selectors Practice Page**

Requirements:

- Demonstrate different selector types: element, class, ID, attribute, and combinators (>, +, ~, descendant).
- Use real HTML elements like paragraphs, lists, and links to show how selectors work.
- Add comments in CSS to describe the purpose of each rule.
- Include a heading and footer to structure the page semantically.
- Maintain clean alignment using padding and margins.

5. **Design a Styled Button Collection**

Requirements:

- Create multiple button styles (primary, secondary, outline, and disabled).
- Use padding, border-radius, and shadows for attractive design.
- Apply hover and active effects using transitions.
- Demonstrate :focus and :disabled pseudo-classes clearly.
- Align buttons using Flexbox and ensure consistent spacing.

6. **Create a CSS Animation Demonstration Page**

Requirements:

- Build at least three animated elements (e.g., moving box, bouncing ball, or color-changing text).
- Use @keyframes to define animations.
- Apply properties like animation-delay, animation-iteration-count, and animation-direction.
- Center animations vertically and horizontally on the page.

- Ensure smooth transitions and clean timing using ease-in-out.

7. Design a User Interface Components Page

Requirements:

- Create a simple UI containing a search bar, buttons, and a dropdown menu.
- Use consistent styling with padding, border-radius, and box-shadow.
- Highlight focused or active elements using `:focus` and `:hover`.
- Keep all components aligned in a card-like container using Flexbox.
- Apply a soft background gradient for an appealing interface.

8. Build a Box-Sizing Comparison Page

Requirements:

- Create two boxes side by side—one using the default box-sizing and one using box-sizing: border-box;.
- Give both boxes equal width and padding to highlight the difference.
- Use contrasting background colors to make comparison clear.
- Include descriptive text explaining how box-sizing affects total size.
- Center the content and ensure responsiveness for smaller screens.

9. Create a Combined Effects Demonstration Page

Requirements:

- Combine pseudo-classes, transitions, and transformations on the same elements.
- For example, make buttons grow slightly and change color on hover.
- Add subtle box-shadow to enhance the effect.
- Use Flexbox to distribute multiple examples evenly across the page.
- Ensure transitions are smooth and visually balanced.

10. **Design a Complete Mini User Interface Layout**

Requirements:

- Build a compact webpage containing a header, a main content card, and a footer.
- Inside the main card, include a form, buttons, and a small image.
- Apply consistent box-sizing, shadows, and border styles.
- Add hover and focus effects using pseudo-classes.
- Use Flexbox for layout alignment and ensure full responsiveness.